

Gas Optimisation Report

Khaled MAQ

April 2026

1 Introduction

Gas optimisation is important in Solidity because storage writes, external calls, dynamic memory operations, and unnecessary bookkeeping all increase execution cost. This report compares a less efficient baseline implementation with an improved implementation for the most expensive function in each contract, using Foundry's `forge test --gas-report`.

2 Contract Comparison: MockGroth16Verifier.sol

Function Optimised: `setShouldVerify`

```
bool[] private _shouldVerifyHistory;  
  
function setShouldVerify(bool shouldVerify)  
    external {  
    _shouldVerify = shouldVerify;  
    _shouldVerifyHistory.push(shouldVerify);  
}
```

Listing 1: Baseline: unnecessary history tracking

```
function setShouldVerify(bool shouldVerify)  
    external {  
    _shouldVerify = shouldVerify;  
}
```

Listing 2: Optimized: direct state update

mocks/MockGroth16Verifier.sol:MockGroth16Verifier Contract					
Deployment Cost	Deployment Size				
361372	1376				
Function Name	Min	Avg	Median	Max	# Calls
setShouldVerify	46348	46348	46348	46348	2
verifyProof	8066	8066	8066	8292	259

Figure 1: Baseline Gas Consumption

mocks/MockGroth16Verifier.sol:MockGroth16Verifier Contract					
Deployment Cost	Deployment Size				
346968	1310				
Function Name	Min	Avg	Median	Max	# Calls
setShouldVerify	21843	21843	21843	21843	2
verifyProof	8066	8066	8066	8292	259

Figure 2: Optimized Gas Consumption

Table 1: Efficiency Gains and Analysis

Function	Old Gas	New Gas	Reduction	Commentary
setShouldVerify	46,348	21,843	-52.9%	The baseline implementation stored unnecessary on-chain history for every verification toggle. The optimized version removes this redundant history array and performs only the required state update, which substantially reduces gas.

3 Contract Comparison: CredentialMetadataRegistry.sol

Function Optimised: registerCredential

```

struct CredentialRecord {
    string issuerDID;
    address issuer;
    uint256 revocationIndex;
    bool exists;
};

CredentialRecord storage record =
    _credentials[credentialId];
if (record.exists) {
    revert CredentialAlreadyRegistered(
        credentialId);
}

record.exists = true;

```

Listing 3: Baseline: explicit existence flag

```

struct CredentialRecord {
    string issuerDID;
    address issuer;
    uint256 revocationIndex;
}

CredentialRecord storage record = _credentials
    [credentialId];
if (record.issuer != address(0)) {
    revert CredentialAlreadyRegistered(
        credentialId);
}

// No extra exists write needed

```

Listing 4: Optimized: reuse issuer as sentinel

src/CredentialMetadataRegistry.sol:CredentialMetadataRegistry contract					
Deployment Cost	Deployment Size				
897368	4318				
Function Name	Min	Avg	Median	Max	# Calls
getCredential	8521	8521	8521	8521	3
getIssuer	2019	2019	2019	2019	259
getIssuerDID	2945	8336	8357	8357	260
getRevocationIndex	2014	2014	2014	2014	259
registerCredential	25618	113331	122677	122677	20

Figure 3: Baseline Gas Consumption

src/CredentialMetadataRegistry.sol:CredentialMetadataRegistry contract					
Deployment Cost	Deployment Size				
905178	4354				
Function Name	Min	Avg	Median	Max	# Calls
getCredential	8521	8521	8521	8521	3
getIssuer	2019	2019	2019	2019	259
getIssuerDID	2945	8336	8357	8357	260
getRevocationIndex	2014	2014	2014	2014	259
registerCredential	25632	93390	100519	100519	20

Figure 4: Optimized Gas Consumption

Table 2: Efficiency Gains and Analysis

Function	Old Gas	New Gas	Reduction	Commentary
registerCredential	113,331	93,390	-17.6%	The baseline implementation used a separate exists flag to track registration, which required an additional storage write. The optimized version reuses the non-zero issuer field as the existence sentinel, eliminating that extra write.

4 Contract Comparison: CredentialVerifier.sol

Function Optimised: `validateCredential`

```
string memory issuerDID =
    credentialMetadataRegistry.getIssuerDID(
        credentialId);
address issuer = credentialMetadataRegistry.
    getIssuer(credentialId);
uint256 revocationIndex =
    credentialMetadataRegistry.
        getRevocationIndex(credentialId);

issuerDID;
```

Listing 5: Baseline: separate getter calls

```
(, address issuer, uint256 revocationIndex) =
    credentialMetadataRegistry.getCredential(
        credentialId);
```

Listing 6: Optimized: consolidated metadata fetch

src/CredentialVerifier.sol:CredentialVerifier Contract					
Deployment Cost	Deployment Size				
1318808	6821				
Function Name	Min	Avg	Median	Max	# Calls
resolveCredentialContext	31669	31669	31669	31669	1
validateCredential	6571	38439	38612	39464	260
verifyPresentationOrRevert	20468	26596	26596	32724	2

Figure 5: Baseline Gas Consumption

src/CredentialVerifier.sol:CredentialVerifier Contract					
Deployment Cost	Deployment Size				
1075910	5680				
Function Name	Min	Avg	Median	Max	# Calls
resolveCredentialContext	33652	33652	33652	33652	1
validateCredential	6510	34536	34692	34924	260
verifyPresentationOrRevert	22451	28579	28579	34707	2

Figure 6: Optimized Gas Consumption

Table 3: Efficiency Gains and Analysis

Function	Old Gas	New Gas	Reduction	Commentary
validateCredential	38,439	34,536	-10.2%	The baseline implementation retrieved data through three separate external getter calls. The optimized version consolidates this into a single metadata fetch, reducing external-call overhead.

5 Contract Comparison: DIDRegistry.sol

Function Optimised: **registerDid**

```
bytes32[] private _registeredDidHashes;

for (uint256 i = 0; i < _registeredDidHashes.
length; i++) {
    if (_registeredDidHashes[i] == didHash) {
        revert DIDAlreadyRegistered(didHash);
    }
}
_registeredDidHashes.push(didHash);
```

Listing 7: Baseline: linear duplicate scan

```
DidDocument storage document = _documents[
    didHash];

if (document.owner != address(0)) {
    revert DIDAlreadyRegistered(didHash);
}
// No auxiliary array push
```

Listing 8: Optimized: direct mapping check

src/DIDRegistry.sol:DIDRegistry Contract					
Deployment Cost	Deployment Size				
1142027	5078				
Function Name	Min	Avg	Median	Max	# Calls
getOwner	3573	3573	3573	3573	19
getPublicKey	6519	6519	6519	6519	1
registerDid	139453	139453	139453	139453	18

Figure 7: Baseline Gas Consumption

src/DIDRegistry.sol:DIDRegistry Contract					
Deployment Cost	Deployment Size				
1069905	4743				
Function Name	Min	Avg	Median	Max	# Calls
getOwner	3573	3573	3573	3573	19
getPublicKey	6519	6519	6519	6519	1
registerDid	94843	94843	94843	94843	18

Figure 8: Optimized Gas Consumption

Table 4: Efficiency Gains and Analysis

Function	Old Gas	New Gas	Reduction	Commentary
registerDid	139,453	94,843	-32.0%	The baseline maintained an extra array and scanned it linearly. The optimized version checks registration directly through the mapping entry, giving constant-time detection.

6 Contract Comparison: RevocationRegistry.sol

Function Optimised: **revoke**

```

if (this.isRevoked(msg.sender, index)) {
    revert AlreadyRevoked(msg.sender, index);
}
(uint256 bucket, uint256 mask) = _position(
    index);
_revocationBitmaps[msg.sender][bucket] =
    _revocationBitmaps[msg.sender][bucket] |
    mask;

```

Listing 9: Baseline: external self-check

```

(uint256 bucket, uint256 mask) = _position(
    index);
uint256 bitmapWord = _revocationBitmaps[msg.
    sender][bucket];

if ((bitmapWord & mask) != 0) {
    revert AlreadyRevoked(msg.sender, index);
}
_revocationBitmaps[msg.sender][bucket] =
    bitmapWord | mask;

```

Listing 10: Optimized: direct bitmap check

src/RevocationRegistry.sol:RevocationRegistry Contract					
Deployment Cost	Deployment Size				
433599	1791				
Function Name	Min	Avg	Median	Max	# Calls
isRevoked	3217	3217	3217	3217	791
revoke	25944	48376	48545	48857	265

Figure 9: Baseline Gas Consumption

src/RevocationRegistry.sol:RevocationRegistry Contract					
Deployment Cost	Deployment Size				
390763	1593				
Function Name	Min	Avg	Median	Max	# Calls
isRevoked	3217	3217	3217	3217	526
revoke	24313	46390	46555	46867	265

Figure 10: Optimized Gas Consumption

Table 5: Efficiency Gains and Analysis

Function	Old Gas	New Gas	Reduction	Commentary
revoke	48,376	46,390	-4.1%	The baseline performed an expensive external self-call. The optimized version performs the duplicate check directly on the local bitmap word, avoiding unnecessary call overhead.

7 Contract Comparison: IdentityRegistry.sol

Function Optimised: registerIdentity

```
address[] private _registeredUsers;
for (uint256 i = 0; i < _registeredUsers.
length; i++) {
    require(_registeredUsers[i] != msg.sender,
        "Already registered");
}
_registeredUsers.push(msg.sender);
```

Listing 11: Baseline: linear duplicate scan

```
require(_identities[msg.sender].status ==
    Status.NotRegistered, "Already registered"
);
// No auxiliary user array push
```

Listing 12: Optimized: direct mapping status check

src/IdentityRegistry.sol:IdentityRegistry Contract					
Deployment Cost	Deployment Size				
1664575	8115				
Function Name	Min	Avg	Median	Max	# Calls
getIdentity	8469	8469	8469	8469	3
isVerified	2956	2956	2956	2956	269
registerIdentity	21971	115571	116409	116409	272
revokeIdentity	36919	36919	36919	36919	1
verifyIdentity	54655	54655	54655	54655	264

Figure 11: Baseline Gas Consumption

src/IdentityRegistry.sol:IdentityRegistry Contract					
Deployment Cost	Deployment Size				
1630091	7954				
Function Name	Min	Avg	Median	Max	# Calls
getIdentity	8469	8469	8469	8469	3
isVerified	2956	2956	2956	2956	269
registerIdentity	21971	71837	72197	72197	272
revokeIdentity	36919	36919	36919	36919	1
verifyIdentity	54655	54655	54655	54655	264

Figure 12: Optimized Gas Consumption

Table 6: Efficiency Gains and Analysis

Function	Old Gas	New Gas	Reduction	Commentary
registerIdentity	115,571	71,837	-37.8%	The baseline scanned an auxiliary array before each registration. The optimized version checks registration status directly from the mapping, significantly lowering storage and compute costs.

8 Contract Comparison: KYCGatedAuction.sol

Function Optimised: placeBid

```
struct BidRecord { address bidder; uint256
    amount; }
BidRecord[] private _bidHistory;
_bidHistory.push(BidRecord({ bidder: msg.
    sender, amount: msg.value }));
```

Listing 13: Baseline: unnecessary on-chain bid history

```
// Removed on-chain bid history storage
highestBidder = msg.sender;
highestBid = msg.value;
emit BidPlaced(msg.sender, msg.value);
```

Listing 14: Optimized: rely on state and events only

src/KYCGatedAuction.sol:KYCGatedAuction Contract					
Deployment Cost	Deployment Size				
1216299	6412				
Function Name	Min	Avg	Median	Max	# Calls
highestBid	2469	2469	2469	2469	259
highestBidder	2530	2530	2530	2530	3
placeBid	35147	142967	144918	144918	267

Figure 13: Baseline Gas Consumption

src/KYCGatedAuction.sol:KYCGatedAuction Contract					
Deployment Cost	Deployment Size				
1181047	6249				
Function Name	Min	Avg	Median	Max	# Calls
highestBid	2469	2469	2469	2469	259
highestBidder	2530	2530	2530	2530	3
placeBid	35147	77565	78326	78326	267

Figure 14: Optimized Gas Consumption

Table 7: Efficiency Gains and Analysis

Function	Old Gas	New Gas	Reduction	Commentary
placeBid	142,967	77,565	-45.7%	The baseline stored unnecessary bid history on-chain. The optimized version removes this redundant tracking and relies on state variables and events, significantly reducing gas.

9 Conclusion

The gas report shows that targeted optimisations can reduce execution cost by removing redundant state writes, avoiding unnecessary external calls, and replacing auxiliary bookkeeping with direct storage checks. The largest improvements were observed in `setShouldVerify` and `placeBid`, where unnecessary on-chain history tracking was removed.